

데이터 유출 방지를 위한 가상 추상화 도구(VAT) 기반 LLM 에이전트 하이브리드 보안 아키텍처

김도영¹, 박준영², 김준식³, 고남현⁴, 윤주범⁵

^{1,2,3,4} 세종대학교 정보보호학과 학부생

⁵ 세종대학교 정보보호학과 교수

dgreezero01@sju.ac.kr, pjy010218@sju.ac.kr, kjs79513@naver.com, glucosamin7@sju.ac.kr, jbyun@sejong.ac.kr

Hybrid Security Architecture Based on Virtually Abstracted Tools (VAT) for LLM Agent Data Loss Prevention

Doyeong Gim¹, Junyeong Park², Junsik Kim³, Namhyun Ko⁴, Ju-Beom Yoon⁵

^{1,2,3,4,5}Dept. of Information Security, Sejong University

요 약

대형 언어 모델(Large Language Model, LLM)을 기반으로 하는 자율형 에이전트 시스템은 모델 문맥 프로토콜(MCP)의 도입과 함께 외부 데이터베이스와의 상호운용성을 극대화하고 있다. 그러나 이러한 발전은 에이전트가 기능적 필요를 초과하여 민감 데이터를 수집하는 데이터 과다 노출(Data Over-Exposure, DOE) 위협과, 무해한 정보 파편을 조합해 민감 정보를 유추해 내는 도구 오케스트레이션 프라이버시 위험(TOP-R)을 야기했다. 이를 방어하기 위해 학계와 산업계는 LLM을 보안 통제 주체로 활용하는 사후 정제(LLM-as-a-Judge) 방식에 의존해 왔으나, 이는 환각(Hallucination) 발생과 지연 시간 증가라는 구조적 한계에 직면해 있다. 본 연구는 이러한 한계를 극복하고 차세대 DLP(Data Loss Prevention) 아키텍처를 구현하기 위해, 최소 권한의 원칙(PoLP)을 설계 단계부터 강제하는 '가상 도구 추상화(Virtually Abstracted Tools, VAT)' 계층 기반의 4 단계 하이브리드 보안 메커니즘을 제안한다. 제안하는 개념적 아키텍처는 에이전트에게 원본 도구를 은닉하고 세분화된 가상 원자 도구만을 노출하여 에이전트의 명시적 선택을 통해 의도를 증명한다. 이후 MCP Server에서 반환된 원본 데이터는 사후 정제를 수행하는 LLM-as-a-Judge에게 전달되지만 이때 에이전트가 선택한 '가상 도구'가 함께 제공된다. 본 논문은 구조적 분석을 통해 이 하이브리드 메커니즘이 LLM-as-a-Judge의 환각을 극적으로 억제하고 지연 시간을 단축할 수 있는 논리적 타당성을 규명한다.

1. 서론

MCP의 등장은 LLM과 외부 시스템 간의 통합을 가속화하였으나, 역설적으로 에이전트가 시스템 수준 권한을 행사하게 만들며 보안 취약점을 야기하고 있다. 현재 가장 치명적인 위협은 레거시 API의 특성상 사용자의 의도와 무관하게 방대한 민감 데이터를 반환하는 광범위 호출(Broad Call)로 인한 DOE 현상이다. 또한, 에이전트가 여러 도구에서 얻은 비민감 정보를 조합해 민감한 결론을 도출하는 도구 오케스트레이션 프라이버시 위험 역시 기존 접근 제어를 무력화하고 있다. 따라서 에이전트 환경에 특화된 새로운 DLP 체계가 요구된다.

이러한 문제를 해결하기 위해 도입된 사후 정제(LLM-as-a-Judge) 방식은 MCP Server에서 원본 데이터를 모두 수신한 뒤 별도의 LLM이 이를 필터링하는

방식이다. 그러나 보안 영역에 비결정론적 확률 모델을 단순 도입하는 것은 모순을 낳는다. 거대한 데이터에서 '무엇이 민감한 정보인지' 모호한 상태로 판단을 위임할 경우 LLM-as-a-Judge는 환각에 의한 데이터 오염을 발생시키며 지연 시간 증가를 초래한다.

본 연구는 사후 검증의 한계를 해결하고 강력한 DLP 파이프라인을 구축하기 위해, 설계 단계에서 데이터 접근을 원천적으로 제어하는 '가상 도구 추상화' 계층을 제안한다.

2. 관련 연구

2.1 데이터 과다 노출(DOE) 및 프라이버시 위험

최근 LLM 에이전트의 도구 상호작용 과정에서 발생하는 DOE의 심각성이 대두된다. AgentRaft 프레임워크 연구에 따르면, 6,675개의 실제 도구 환경에서

57.07%의 잠재적 상호작용 경로가 기능적 필요를 초과한 민감 데이터를 노출하는 것으로 확인되었다[1]. 또한, 개별 도구의 응답 자체는 안전하더라도 이를 에이전트가 조합하여 민감 정보를 재구성하는 TOP-R 위협 역시 90.24%의 높은 위험 유출률을 기록하며 다중 도구 오케스트레이션 환경의 치명적 한계를 드러냈다[2].

2.2 LLM-as-a-Judge 시스템의 한계

데이터 유출을 막기 위해 LLM-as-a-Judge 를 검증 필터로 배치하는 사후 필터링 방식이 주로 논의되어 왔다. 그러나 최근 연구들은 LLM-as-a-Judge 시스템이 복잡한 데이터 구조에서 어떤 정보를 남기고 삭제할지 스스로 판단해야 하므로 구조적인 환각에 취약하며, 평가 후보의 순서에 따라 결과가 달라지는 위치 편향(Position Bias) 등 비결정론적 한계를 내포하고 있다고 지적한다[3]. 따라서 AI 에이전트의 보안은 단순히 LLM 에게 모호한 판단을 전가하는 방식을 넘어 시스템 아키텍처 차원에서 명확한 판단 기준을 사전에 추출하여 LLM 에게 엄격한 제약으로 제공하는 하이브리드 아키텍처 방향으로 논의되고 있다[4].

3. VAT 아키텍처 설계

VAT 계층은 프록시 레이어에서 에이전트와 MCP Server 사이의 통신을 중재하는 4 단계 DLP 파이프라인으로 구성된다. MCP Server 에는 특정 직원의 연봉만을 요구해도 부서나 연락처 등 전체 인사 데이터를 반환하는 도구(광범위 호출 도구)만 존재할 수 있다. 반면 프록시 계층이 에이전트에게 노출하는 도구 집합은 오직 단일 목적만을 수행하는 원자적 도구들이다. 프록시는 이 둘 사이의 번역기 역할을 수행한다. 에이전트가 가상 도구인 `seek_salary` 를 호출하면 프록시는 내부적으로 이 요청을 실제 MCP Server 의 `get_all_hr_data` 로 치환하여 1:1 매핑한다. 이와 동시에 추후 데이터를 정제할 LLM-as-a-Judge 에게 Agent 가 선택한 원자적 도구를 전달한다. 이러한 구조를 바탕으로 다음과 같은 4 단계 파이프라인이 실행된다.

1 단계: 도구 목록 변조 (Tool List Spoofing)

에이전트가 사용 가능한 도구 목록을 파악하기 위해 `tools/list` 요청을 전송하면 프록시가 이를 가로챈다. 기존의 도구 리스트(예: `get_all_hr_data`)는 은닉되며 사전에 정의된 원자적 도구 리스트(예: `seek_salary_column`)만을 에이전트에게 응답한다.

2 단계: 가상의 툴 콜 발생 및 의도 증명

에이전트는 변조된 도구 목록 내에서 가장 적합한 도구를 호출한다. 이 단계의 핵심은 에이전트의 구체적인 툴 콜 자체가 사용자의 의도를 증명하는 결정적

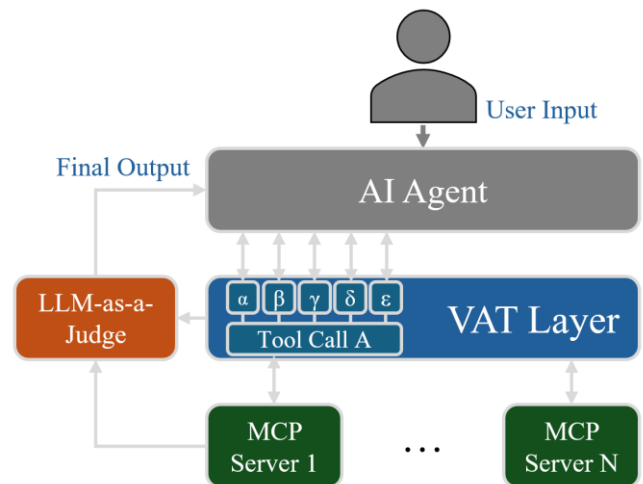
식별표로 작용한다는 점이다. 에이전트가 원자적 도구를 스스로 선택했으므로 시스템은 에이전트의 목표 지향성을 사전에 명확히 파악할 수 있다.

3 단계: 실제 툴 콜 인터셉트 및 MCP Server 변환 호출

에이전트가 특정 가상 도구를 선택해 실행(Tool Call)하면 프록시는 이 요청이 실제 MCP Server 로 바로 전달되지 않도록 중간에서 가로챈다. 이후 프록시는 내부 매트릭스를 확인하여 에이전트가 고른 가상 도구와 연결된 실제 MCP Server 의 도구를 찾아낸다. 프록시는 에이전트를 대신하여 이 도구를 실제 MCP Server 에 정상적으로 호출한다.

4 단계: 의도 기반 사후 정제 및 최종 반환

프록시는 수신한 데이터와 함께 2 단계에서 확보한 가상 도구를 LLM-as-a-Judge 에게 제공한다. 원래의 툴 콜로 인해 반환된 데이터와 함께 에이전트가 선택한 원자 도구를 줌으로써 LLM-as-a-Judge 는 원본 데이터에서 필요한 부분만 추출하며, 정제된 원자적 데이터만이 에이전트에게 최종 반환된다.



(그림 1) 아키텍처 계층도

4. 기존 모델과의 구조적 비교 및 기대 효과

4.1 의도 지시를 통한 환각 억제 및 DOE 해소

기존 사후 정제 방식은 방대한 데이터를 수신한 후 검증 모델에게 "민감 정보를 찾아 절삭하라"고 모호하게 지시한다. 이 과정에서 모델은 복잡한 데이터 구조의 맥락을 백지상태에서 추론해야 하므로 엉뚱한 데이터를 남기거나 문법 오류를 범하는 환각이 빈번하게 발생한다. 반면 VAT 메커니즘은 LLM-as-a-Judge 에게 원본 데이터와 함께 가상 도구 스키마라는 제약 조건을 부여한다. LLM 은 데이터를 주관적으로 평가할 필요 없이 주어진 가이드라인에 부합하는 값만 매칭하여 추출하는 작업만을 수행한다. 판단의 자율성을 제한하고 명확한 기준을 제시함으로써 LLM 의 환

각 발생률은 이론적으로 감소한다.

4.2 도구 오케스트레이션 프라이버시 위험 해소

사후 필터링이 불완전하게 작동할 경우 컨텍스트 윈도우 내에 불필요한 정보 파편이 남겨지고, 이는 에이전트의 교차 추론을 돕는 TOP-R 위협의 핵심 단서로 악용된다. 그러나 VAT 프록시는 하이브리드 정제 계층을 거쳐 완벽히 가공된 단일 원자 데이터만을 에이전트에게 주입한다. 주변부 정보 자체가 인지 범위 내에 유입되지 못하게 원천 차단함으로써 에이전트는 모자이크 추론에 필요한 조각을 물리적으로 박탈당하게 되어 TOP-R 위협 성립 자체를 논리적으로 무력화할 수 있다.

4.3 연산 부하 오프로딩 및 지연 시간 단축

기존 LLM-as-a-Judge 는 데이터를 프롬프트로 처리하고 필터링 로직을 텍스트로 추론 및 생성하는 과정에서 막대한 토큰 소모와 지연 시간을 유발한다. 반면, 제안하는 시스템은 LLM-as-a-Judge 의 추론 부하를 낮춰 지연 시간 문제를 해결한다. LLM 은 방대한 텍스트로 보안 논리를 스스로 생성할 필요 없이 프록시가 제시한 가상 톨의 규격에 맞는 짧은 JSON 데이터만 출력하면 되기 때문에 출력 토큰 수가 최소화되고 처리 속도가 상승한다.

5. 결론 및 향후 과제

AI 에이전트의 동적 도구 상호작용은 제어 불가능한 데이터 과다 노출과 도구 오케스트레이션 프라이버시 위험을 수반한다. 본 연구는 정보 보안 업계가 직면한 모호한 확률적 검증의 한계를 해결하고 데이터 유출을 방지하기 위한 VAT 계층 기반 하이브리드 보안 아키텍처를 제안하였다.

제안된 메커니즘은 도구 목록 변조를 통해 에이전트의 실행 환경을 최소 권한의 원칙[5]에 종속시키며, 여기서 증명된 에이전트의 명확한 의도를 LLM-as-a-Judge 에게 정답지처럼 제공한다. 이 구조는 환각의 개입을 차단하고 지연 시간을 크게 단축할 수 있는 잠재력을 지닌다.

5.1 한계점

본 연구가 제안하는 VAT 계층은 강력한 사전 보안 통제를 제공하지만 실제 엔터프라이즈 환경 적용을 고려할 때 구조적 한계를 지닌다.

첫째, 병렬 도구 호출로 인한 병목 현상이다. 에이전트가 복수의 가상 원자 도구를 동시에 호출할 경우 프록시는 논리적으로 동일한 원본 도구를 중복으로 반복 호출하게 되어 서버 과부하와 실행 비효율을 유발할 수 있다[6].

둘째, 가상 톨 목록을 누가 어떻게 생성할 것인가에 대한 문제이다. 기존 톨에 필터링 함수를 단순히 추가하는 방식을 넘어, 존재하지 않는 가상 톨의 스키마를 새롭게 설계하고 매핑해야 하므로 초기 구축

에 막대한 인적, 시간적 자원이 소요된다.

5.2 향후 연구 방향

위와 같은 한계를 극복하고 제안된 아키텍처를 실증적 시스템으로 발전시키기 위해 다음과 같은 향후 연구가 수반되어야 한다.

첫째, MCP Server 과부하 방지를 위한 캐싱 및 요청 병합 메커니즘의 구현이다. 동일 세션 내에서 요청되는 다수의 가상 도구 호출을 논리적으로 병합하여 단일 MCP Server 호출로 처리하고 실시간성이 중요한 데이터의 캐시 일관성을 보장하는 라우팅 최적화 연구가 필요하다.

둘째, OpenAPI 스키마 등 기존 MCP Server API 명세서를 분석하여 도구 목록을 표준화된 명세로 정제하고[7], 이를 바탕으로 광범위 도구를 식별하여 대응하는 가상 원자 도구 매트릭스를 구축하는 파이프라인 연구가 요구된다.

셋째, 노출된 도구 목록을 통한 시스템 구조 유출을 방지하기 위해 가상 도구의 식별자를 난독화하는 기법의 도입이 필요하다.

향후 벤치마크 테스트를 통한 실증적 성능 평가가 수반된다면 본 연구는 레거시 시스템의 MCP Server 수정 없이도 안전한 AI 거버넌스를 가능하게 할 것이다.

참고문헌

- [1] Y. Lin, J. Wu, Y. Nan, X. Wang, X. Zhang, and Z. Zheng, "AgentRaft: Automated Detection of Data Over-Exposure in LLM Agents," arXiv preprint arXiv:2603.07557, 2026.
- [2] Y. Qiao, D. Liu, H. Yang, W. Zhou, and S. Hu, "Agent Tools Orchestration Leaks More: Dataset, Benchmark, and Mitigation," arXiv preprint arXiv:2512.16310, 2025.
- [3] L. Shi, C. Ma, W. Liang, X. Diao, W. Ma, and S. Vosoughi, "Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge," arXiv preprint arXiv:2406.07791, 2024.
- [4] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions," arXiv preprint arXiv:2503.23278, 2025.
- [5] K. Zhang, Z. Su, P.-Y. Chen, E. Bertino, X. Zhang, and N. Li, "LLM Agents Should Employ Security Principles," arXiv preprint arXiv:2505.24019, 2025.
- [6] J. Shen, N. Wadlom, and Y. Lu, "Batch Query Processing and Optimization for Agentic Workflows," arXiv preprint arXiv:2509.02121, 2025.
- [7] S. Yuan, K. Song, J. Chen, X. Tan, Y. Shen, R. Kan, D. Li, and D. Yang, "EASYTOOL: Enhancing LLM-based Agents with Concise Tool Instruction," arXiv preprint arXiv:2401.06201, 2024.